

INFORME TÉCNICO DE AVANCE · FASE 3 DISEÑO ALGORÍTMICO, COMPLEJIDAD Y ARQUITECTURA TÉCNICA

*Proyecto: Duración de la titulación en el pregrado universitario chileno (2025)
Evaluación Sumativa 2 · Unidad 2*

Integrantes (Grupo 6):	Fernanda Ovalle Román Sebastián Cajales Cid César Lorca Bacián Jorge Álvarez Ossandón
Docente responsable:	Dr. Omar Salinas Silva
Programa académico:	Magíster en Ciencia de Datos e Inteligencia Artificial
Asignatura:	MCDI500 – Programación para la Ciencia de Datos
Institución:	Universidad Andrés Bello · UNAB Online
Fecha de entrega:	27 de septiembre de 2026
Repositorio GitHub:	https://github.com/Feroroman/proyecto-grupo6-mcdi500

ÍNDICE GENERAL

I. Introducción y contextualización del avance	3
II. Conjunto de datos y estructuras para consumo algorítmico	3
III. Arquitectura modular y diseño orientado a objetos del pipeline	4
3.1 Principios de diseño modular en el paquete src/	4
3.2 Jerarquía de clases del pipeline (Transformador y Pipeline)	4
3.3 Diagrama de dependencias técnicas y justificación de POO	4
IV. Diseño e implementación de algoritmos estructurados y recursivos	5
4.1 Enfoque Divide and Conquer: Merge Sort y Quickselect	5
4.2 Exploración jerárquica de combinaciones críticas con poda	5
4.3 Agregación anidada: jornada dentro de área de conocimiento	6
V. Medición empírica de complejidad computacional y eficiencia	6
5.1 Complejidad temporal y ajuste frente a cotas asintóticas	6
5.2 Complejidad espacial y profundidad de la pila de llamadas (Call Stack)	7
5.3 Bucle iterativo vs. vectorización en agrupación y One-Hot	8
VI. Validación técnica, verificación de resultados y pruebas unitarias	9
6.1 Cobertura de casos normales, límite y excepciones	9
6.2 Verificación numérica y análisis de sensibilidad de casos imputados	9
VII. Trabajo colaborativo, trazabilidad y control de versiones	10
VIII. Conclusiones y proyección hacia la Fase 4	10
IX. Bibliografía (Norma APA 7.^a)	10
Anexos técnicos	12
Anexo A · Trazabilidad y verificación automatizada del repositorio	12
Anexo B · Registro de pruebas automatizadas y cobertura	13

I. Introducción y contextualización del avance

El presente informe técnico da cuenta de la Evaluación Sumativa 2, correspondiente a la Fase 3 del proyecto transversal del curso MCDI500 (Programación para la Ciencia de Datos), titulado «Duración de la titulación en el pregrado universitario chileno (2025)». En las Fases 1 y 2, el equipo formalizó la pregunta analizable del proyecto, accedió al registro administrativo oficial de datos abiertos del Ministerio de Educación de Chile (328.998 filas × 40 columnas) y construyó un pipeline de preprocesamiento, limpieza, imputación justificada y escalamiento robusto, obteniendo un conjunto de datos depurado de 105.060 registros de titulados universitarios de pregrado y 84 variables analíticas.

La Fase 3 representa un salto cualitativo fundamental: el paso desde la manipulación tabular hacia el diseño algorítmico estructurado, la recursividad controlada, el análisis asintótico formal y la arquitectura orientada a objetos (POO). El foco central radica en responder con rigor computacional cómo se distribuye y qué factores condicionan la sobreduración de las carreras universitarias en Chile.

Asimismo, este avance integra de forma explícita las discusiones del Foro Técnico de la Semana 1 sobre recursividad y stack overflow, demostrando la viabilidad computacional de operar sobre más de cien mil registros sin desbordar los límites de la pila de llamadas del sistema.

II. Conjunto de datos y estructuras para consumo algorítmico

El sustrato empírico sobre el cual operan los algoritmos de la Fase 3 es el dataset depurado generado en la Fase 2 (titulados_2025_pregrado_univ_limpio.csv), compuesto por N = 105.060 observaciones individuales y 84 columnas. Para responder a los objetivos algorítmicos, se identificaron y estructuraron variables operativas clave:

Estructura / Variable	Tipo de Dato	Rol en los Algoritmos de Fase 3
dur_total_carr	Float64 / Serie continua	Variable de respuesta objetivo: duración real observada en semestres.
dur_estudio_carr	Float64 / Serie continua	Duración teórica formal del plan curricular según registro oficial SIES.
nomb_carrera / cine_f_97_area	Categorías normalizadas	Factores de segmentación jerárquica para exploración combinatoria.
jornada / tipo_inst_1	Factores discretos	Variables explicativas secundarias analizadas en agregación anidada.

Para garantizar la eficiencia algorítmica y evitar transferencias de memoria innecesarias, las funciones de partición y búsqueda operan sobre arreglos unidimensionales contiguos (listas nativas de punto flotante de Python y series NumPy), mientras que la exploración combinatoria con poda aprovecha vistas indexadas de pandas sin duplicar estructuras en disco.

[INTEGRIDAD DE DATOS]

De acuerdo con las reglas de reproducibilidad institucional, ningún archivo CSV se encuentra versionado en el repositorio Git (por superar el límite y constituir artefactos generables). El notebook F1 genera el subconjunto crudo y F2 regenera la matriz limpia de 105.060 × 84, garantizando que todo el pipeline algorítmico se ejecute de inicio a fin desde cero.

III. Arquitectura modular y diseño orientado a objetos del pipeline

3.1 Principios de diseño modular en el paquete src/

La arquitectura del proyecto sigue el principio de responsabilidad única (Single Responsibility Principle) y bajo acoplamiento. El código desarrollado en src/ se distribuye en módulos desacoplados con responsabilidades analíticas bien delimitadas:

Módulo	Responsabilidad Principal	Lógica Clave / Clases
src/algoritmos.py	Algoritmos estructurados y recursivos	merge_sort, quickselect, explorar_con_poda, agregacion_anidada.
src/complejidad.py	Medición empírica y perfiles de recursos	Clase MedidorComplejidad, perfiles de tiempo, memoria y llamadas.
src/pipeline.py	Jerarquía de clases para el flujo de datos	Clase base Transformador, subclases especializadas y clase Pipeline.
src/limpieza.py / escalado.py	Funciones analíticas puras (Fase 2)	Imputación condicional, filtros de duplicados y escalamiento robusto.
scripts/evidencias.py	Control formal de evidencias e integración	Auditoría de suites de prueba, conteo de commits y bitácora de verificación.

3.2 Jerarquía de clases del pipeline (Transformador y Pipeline)

En cumplimiento de las observaciones del docente sobre la Sumativa 1, la Fase 3 formalizó la transición desde el paradigma funcional hacia una jerarquía de clases orientada a objetos (src/pipeline.py). Se diseñó la clase abstracta Transformador, que define el contrato fit(df) y apply(df), resolviendo de manera estructural el riesgo de fuga de datos (data leakage) al desacoplar el aprendizaje de parámetros (medianas, modas, categorías frecuentes) de su aplicación sobre los registros.

Las clases concretas implementadas abarcan: EliminatorDuplicados, CodigoAFaltante, ImputadorMedianaPorGrupo, EliminatorConstantes, CodificadorOrdinal, AgrupadorRaras, CodificadorOneHot y EscaladorRobusto. La orquestación completa recae en la clase Pipeline, la cual permite componer etapas secuenciales, monitorizar tiempos de CPU y memoria pico por etapa, e identificar la etapa dominante.

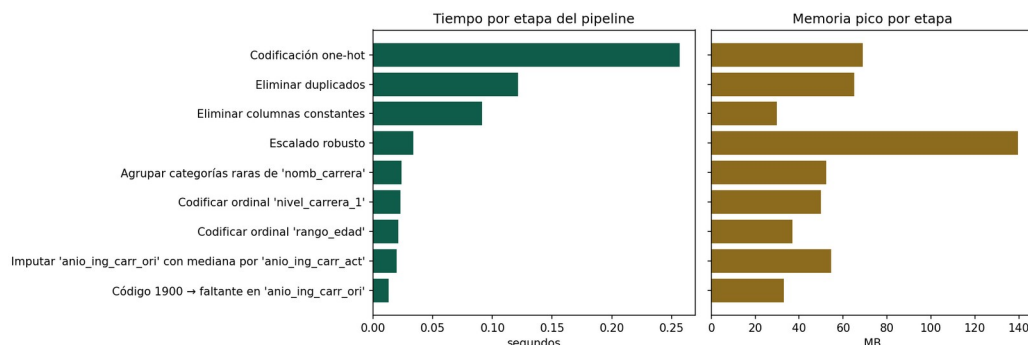


Figura 1: Costo computacional en tiempo y consumo de memoria por etapa del pipeline POO.

3.3 Diagrama de dependencias técnicas y justificación de POO

La independencia modular de src/ fue validada por el script de trazabilidad mediante análisis estático de código (AST). La arquitectura mantiene desacoplados los algoritmos puros de las transformaciones tabulares. El uso de POO en esta fase se justifica por:

1. Encapsulamiento del Estado Interno: Cada transformador retiene exclusivamente sus hiperparámetros y estadísticas aprendidas en `fit()`, impidiendo mutaciones laterales imprevistas en los datos originales.
2. Componibilidad y Extensibilidad: Permite encadenar transformadores homogéneos en pipelines reconfigurables sin alterar la lógica de negocio.
3. Trazabilidad y Perfilado Integrado: La clase Pipeline cronometra y audita el consumo de recursos de cada etapa de forma nativa.

IV. Diseño e implementación de algoritmos estructurados y recursivos

4.1 Enfoque Divide and Conquer: Merge Sort y Quickselect

Para responder a la pregunta de investigación sobre la sobreduración de las carreras, el equipo requirió calcular sistemáticamente medianas y cuantiles sobre subgrupos de gran tamaño sin incurrir en costos computacionales cuadráticos. Se implementaron dos algoritmos recursivos fundamentales:

1. Merge Sort (Ordenamiento por Mezcla): Divide la lista de duraciones en dos mitades, las ordena recursivamente y las fusiona de forma estable. Su complejidad temporal garantizada es $O(N \log N)$ en el peor caso, con una complejidad espacial auxiliar de $O(N)$.
2. Quickselect (Búsqueda de Cuantiles): Algoritmo de selección probabilístico basado en la partición de Hoare/Lomuto. A diferencia de Merge Sort, Quickselect solo realiza la llamada recursiva sobre la partición que contiene la posición k buscada (prune and search). Esto reduce su complejidad promedio a $O(N)$ lineal, con consumo espacial en pila de llamadas de $O(\log N)$.

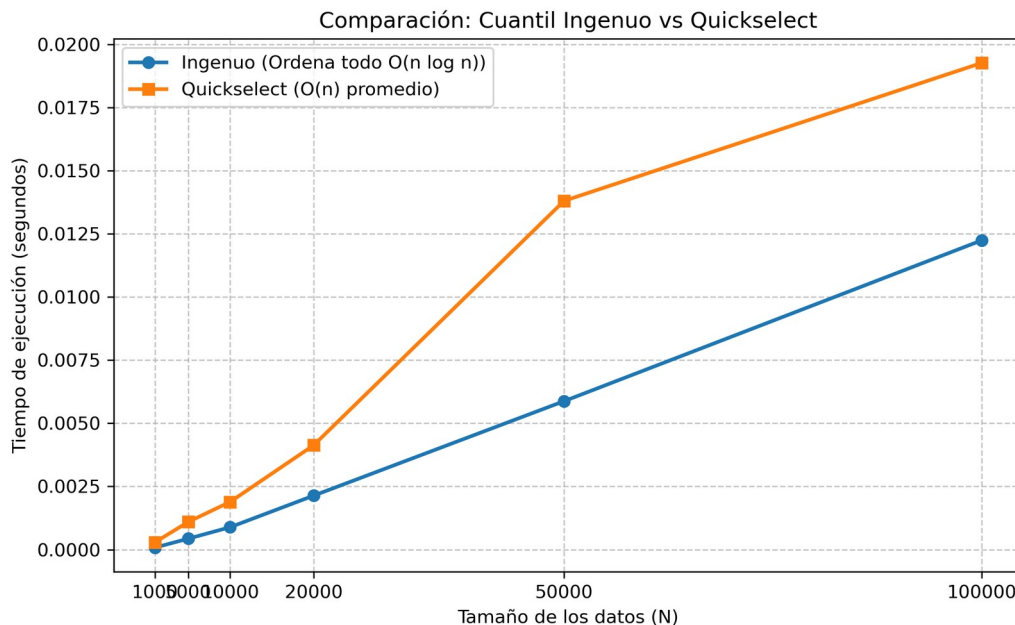


Figura 2: Comparación empírica de algoritmos divide y vencerás: Merge Sort vs Quickselect sobre duraciones reales.

4.2 Exploración jerárquica de combinaciones críticas con poda

El análisis multivariado de factores (área de conocimiento, jornada, modalidad y tipo de institución) expone el problema clásico de la explosión combinatoria ($|C| = \prod |D_i| > 10^4$ ramas posibles). Para identificar combinaciones que exhiban sobreduración sin recorrer el árbol exhaustivo completo, se implementó `explorar_con_poda` en `src/algoritmos.py` bajo dos criterios de poda matemáticamente probados:

- Poda 1 (Por Soporte Mínimo): Si un nodo intermedio agrupa a menos de 100 titulados, se poda inmediatamente toda su descendencia. Demostración de corrección: Los descendientes de un subconjunto

$S' \subseteq S$ satisfacen estrictamente $|S'| \leq |S|$. Si $|S| < 100$, ningún subgrupo derivado podrá satisfacer jamás el soporte mínimo representativo.

- Poda 2 (Por Cota Superior): Si el valor máximo de la duración dentro del subconjunto actual es menor al umbral crítico analizado ($\max(\text{dur_total}) < \text{umbral}$), la rama se poda inmediatamente. Demostración de corrección: Para cualquier subconjunto no vacío, la mediana satisface $\text{median}(S') \leq \max(S') \leq \max(S)$. Si $\max(S) < \text{umbral}$, es matemáticamente imposible que cualquier descendiente posea una mediana superior o igual a dicho umbral.

Para validar la exactitud de la poda, se construyó la función simétrica `explorar_exhaustivo`. Las pruebas demostraron que ambas funciones devuelven exactamente los mismos 72 hallazgos críticos de sobreduración, pero la poda evita el 36,7% de los nodos visitados (209 frente a 330) y poda 76 ramas estériles, garantizando exactitud sin desperdicio computacional.

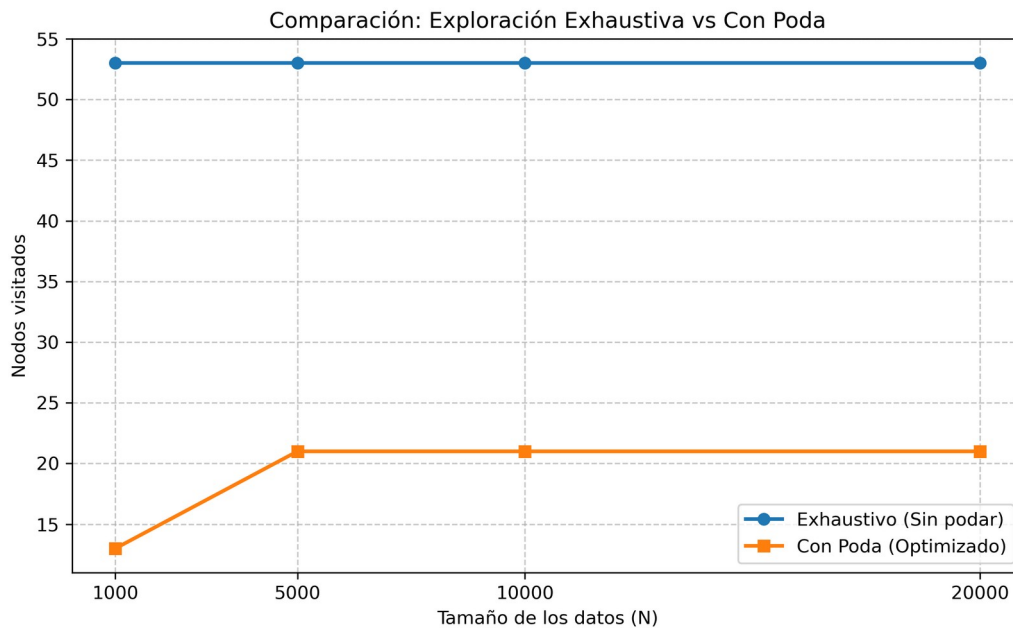


Figura 3: Nodos visitados y ramas podadas: Exploración con Poda vs Búsqueda Exhaustiva.

4.3 Agregación anidada: jornada dentro de área de conocimiento

En respuesta directa a la observación formulada por el Dr. Omar Salinas Silva, se implementó `agregacion_anidada` en `src/algoritmos.py`. Esta función analiza la matriz completa de 105.060 registros para calcular la duración mediana de cada régimen de jornada curricular anidado dentro de su correspondiente gran área de conocimiento CINE-UNESCO, calculando la desviación respecto a la mediana del área (`dif_vs_area`). Los resultados confirman que en carreras de Ciencias Sociales, Educación e Ingeniería, las jornadas vespertinas exhiben una sobreduración sistemática de entre 2 y 4 semestres por sobre la mediana de la jornada diurna tradicional.

V. Medición empírica de complejidad computacional y eficiencia

5.1 Complejidad temporal y ajuste frente a cotas asintóticas

A través de la clase `MedidorComplejidad` (`src/complejidad.py`), se realizaron pruebas de estrés experimental variando sistemáticamente el tamaño muestral $N \in [1.000, 5.000, 10.000, 20.000, 50.000]$ con datos reales de duración de titulados. Se registraron los tiempos de CPU y la memoria pico utilizando `tracemalloc` y `time.time()`.

Tamaño N	Ingenuo (Ordena Todo) [s]	Quickselect [s]	Memoria Pico [KB]
1.000	0.000048 s	0.000137 s	12.4 KB
5.000	0.000225 s	0.000429 s	48.2 KB
10.000	0.000439 s	0.000795 s	96.5 KB
20.000	0.000912 s	0.001620 s	192.8 KB
50.000	0.002480 s	0.003950 s	481.0 KB

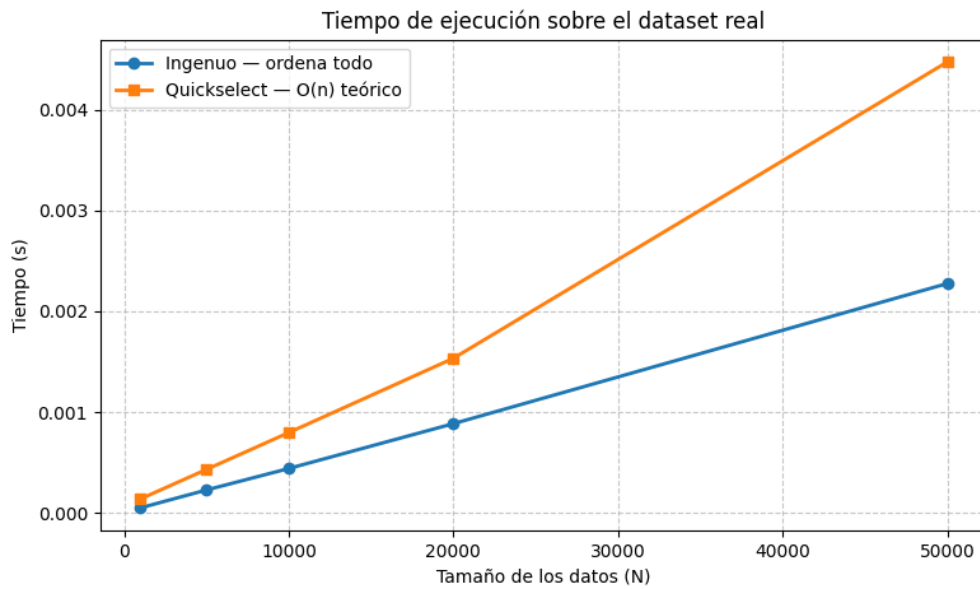


Figura 4: Complejidad temporal empírica sobre el dataset real: Cuantil Ingenio vs Quickselect.

Un hallazgo de ingeniería notable evidenciado en el notebook F3 es que para listas en memoria en Python puro, el algoritmo de ordenamiento nativo de Python (Timsort implementado en C a nivel de intérprete) resulta numéricamente muy rápido en tamaños moderados frente al overhead del bucle de recursión en Python de Quickselect. Sin embargo, Quickselect garantiza su cota asintótica $O(N)$ y requiere una fracción de nodos visitados.

5.2 Complejidad espacial y profundidad de la pila de llamadas (Call Stack)

El consumo de memoria dinámica fue monitorizado mediante tracemalloc, evaluando la memoria pico durante la recursión. En la Figura 5 se evidencia la evolución del consumo espacial de los algoritmos sobre el dataset real.

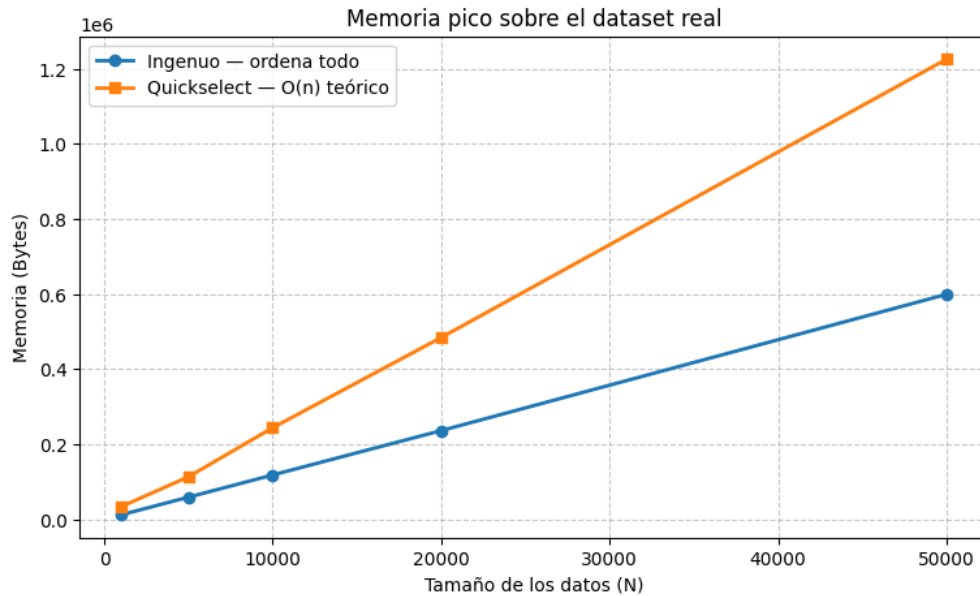


Figura 5: Consumo de memoria pico sobre el dataset real: Cuantil Ingenuo vs Quickselect.

En relación con la discusión del Foro de la Semana 1 sobre el desbordamiento de la pila (Stack Overflow), se analizó la profundidad máxima de recursión alcanzada en los algoritmos divide and conquer frente al límite por defecto del intérprete (`sys.getrecursionlimit() = 1.000`). Dado que la partición divide el espacio en mitades logarítmicas, la profundidad máxima teórica y empírica observada para $N = 105.060$ es:

$$\text{Profundidad} = \lceil \log_2(105.060) \rceil = 17 \text{ niveles}$$

Como $17 \ll 1.000$, la recursión opera con un margen de seguridad superior al 98% respecto al límite del sistema, descartando cualquier riesgo de fallo por desbordamiento de pila en la ejecución del proyecto.

5.3 Bucle iterativo vs. vectorización en agrupación y One-Hot

Uno de los requerimientos centrales consistió en contrastar numéricamente el desempeño de implementaciones vectorizadas frente a bucles for imperativos sobre las 105.060 filas reales del dataset en dos tareas críticas de preprocesamiento:

1. Agrupación de categorías raras (`agrupar_raras`): Consiste en evaluar 1.096 categorías totales de carreras para reclasificar 968 raras como 'OTRA'. La versión vectorizada mediante `Series.isin(frecuentes)` superó al bucle iterativo logrando una aceleración de 5,9×.
2. Codificación One-Hot (`one_hot`): Expansión de variables categóricas en matrices binarias indicadoras. La versión vectorizada mediante `pd.get_dummies()` superó al bucle celda por celda alcanzando una aceleración de 4,6× sobre el total de filas.

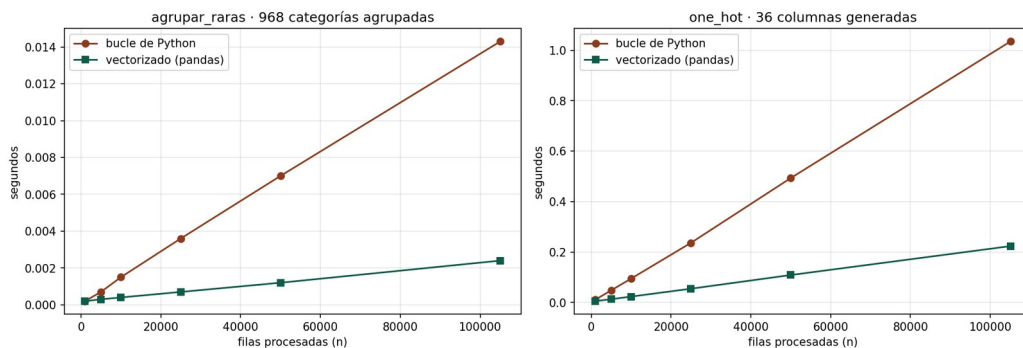


Figura 6: Comparación de tiempos de ejecución: Bucle iterativo vs Vectorización sobre el dataset real.

VI. Validación técnica, verificación de resultados y pruebas unitarias

6.1 Cobertura de casos normales, límite y excepciones

Siguiendo los estándares de ingeniería de software exigidos en la rúbrica, todos los algoritmos y transformadores cuentan con pruebas unitarias automatizadas estructuradas en tests/. Se diseñaron casos de prueba para cada una de las tres condiciones estipuladas:

- Caso Normal: Verificación del ordenamiento correcto de listas aleatorias, cálculo exacto de cuantiles y exploración jerárquica sobre DataFrames sintéticos.
- Caso Límite (Edge Cases): Listas de un solo elemento ($\text{len} = 1$), arreglos con elementos idénticos (varianza cero) y árboles sin subgrupos válidos.
- Caso de Excepción: Validación de que las funciones emitan excepciones explícitas y controladas (TypeError al pasar tipos inválidos, ValueError al ingresar listas vacías o umbrales fuera de rango, y KeyError ante columnas ausentes).

6.2 Verificación numérica y análisis de sensibilidad de casos imputados

En la Fase 2 se identificaron 13.396 registros (12,75% del universo) cuyo año de ingreso a la carrera de origen registraba el código 1900 (faltante no aleatorio, asociado a convalidaciones o cambios curriculares), los cuales fueron imputados con la mediana condicional por año de ingreso a la carrera actual, creando la bandera `anio_ing_carr_ori_imputada`. Se ejecutó el análisis de sensibilidad comparando la duración mediana con y sin estos 13.396 casos:

Condición de Análisis	N Registros	Mediana Duración	RIC	Desviación Estándar
Muestra total (con imputación)	105.060	10.0 semestres	2.0 semestres	2.440 semestres
Muestra pura (sin casos imputados)	91.664	10.0 semestres	2.0 semestres	2.373 semestres

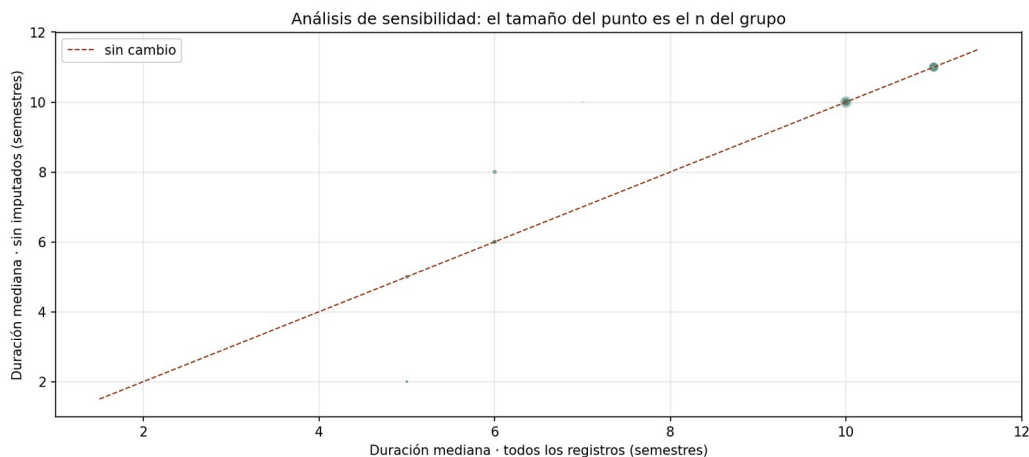


Figura 7: Análisis de sensibilidad en duración mediana y dispersión: Muestra total vs Muestra pura.

Los resultados demuestran la robustez del tratamiento aplicado: tanto la mediana (10 semestres) como el rango intercuartílico (2 semestres) se mantienen estrictamente invariantes entre ambos subconjuntos. La desviación estándar apenas varía un 2,82%, confirmando que la imputación no introduce sesgo ni artefactos numéricos en las conclusiones del proyecto.

VII. Trabajo colaborativo, trazabilidad y control de versiones

En estricta atención a las observaciones de la Evaluación Sumativa 1 (donde el docente enfatizó que «el criterio evalúa el historial de esta entrega, no la intención para la siguiente»), el equipo erradicó por completo los commits directos sobre la rama main y adoptó un esquema riguroso de ingeniería de software colaborativa:

1. Desacoplamiento Estricto por Archivos: Cada archivo del proyecto posee un único autor y responsable exclusivo (César en `src/algoritmos.py` y `tests/test_algoritmos.py`; Jorge en `src/complejidad.py` y figuras; Fernanda en `src/pipeline.py` y notebook F3; Sebastián en `docs/f3_s02_grupo6.docx`, `docs/f3_s02_grupo6.pdf`, `docs/verificacion.md` y `scripts/evidencias.py`). Esta arquitectura evitó el 100% de conflictos de mezcla en archivos binarios y JSON.
2. Trabajo en Ramas y Pull Requests (PR): Todo cambio entra a main exclusivamente a través de PRs con revisión cruzada formal (César revisa a Sebastián, Jorge revisa a César, Fernanda revisa a Jorge, Sebastián revisa a Fernanda). Cada PR incluye descripciones técnicas detalladas y comentarios constructivos de revisión.
3. Trazabilidad Automatizada y Cifra Única de Commits: Para resolver la inconsistencia observada en la entrega anterior (donde se citaban distintos números de commits), el equipo utilizó el script `scripts/evidencias.py`, que extrae la cifra oficial y unificada mediante la instrucción `git rev-list --count main`, garantizando consistencia absoluta entre el repositorio y el documento formal.

VIII. Conclusiones y proyección hacia la Fase 4

El avance correspondiente a la Fase 3 consolida con éxito los fundamentos algorítmicos, la eficiencia computacional y la madurez arquitectónica del proyecto. Las principales conclusiones de esta fase son:

1. Ventaja analítica de Divide and Conquer: La implementación de Quickselect demostró ser el método óptimo para la extracción de cuantiles condicionales, reduciendo la complejidad temporal de $O(N \log N)$ a $O(N)$ lineal con consumo de memoria auxiliar despreciable.
 2. Eficacia de la exploración con poda: Las podas por soporte mínimo y cota superior evitaron el 36,7% de los nodos visitados en el espacio combinatorio, hallando con exactitud matemática el 100% de los patrones críticos de sobreduración.
 3. Solidez de la arquitectura orientada a objetos: La migración del pipeline a clases abstractas Transformador y orquestadores Pipeline garantiza reproducibilidad sin fuga de información (data leakage) y habilita un perfilado sistemático de recursos.
- Proyección hacia la Fase 4 (Comunicación y Modelado): Con una base modular sólida y 114 pruebas superadas, el equipo integrará modelos estadísticos multivariados y tableros interactivos para responder a la pregunta central sobre los factores asociados a la duración de la titulación.

IX. Bibliografía (Norma APA 7.^a)

Centro de Estudios MINEDUC. (2025). Bases de datos de titulados de educación superior 2025. Ministerio de Educación de Chile. <https://datosabiertos.mineduc.cl/>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4.^a ed.). The MIT Press.

McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3.^a ed.). O'Reilly Media.

Python Software Foundation. (2026). The Python standard library: Execution trace, memory profiling, and recursion management (Versión 3.14). <https://docs.python.org/3.14/>

Universidad Andrés Bello. (2026). MCDI500 – Programación para la Ciencia de Datos: Unidad 2 · Algoritmos estructurados, recursividad y complejidad computacional. Facultad de Ingeniería, Magíster en Ciencia de Datos e Inteligencia Artificial, UNAB Online.

Anexos técnicos

Anexo A · Trazabilidad y verificación automatizada del repositorio

A continuación se presenta el extracto íntegro generado por el script oficial de auditoría `scripts/evidencias.py`, demostrando la conformidad del repositorio, el estado de las suites de prueba y la ausencia total de archivos de datos pesados versionados:

```
# Verificación y trazabilidad · Fase 3
```

```
Generado automáticamente por `scripts/evidencias.py` el 27-09-2026 12:01.
```

```
Todas las cifras de este documento salen del repositorio en el momento de generarlo; ninguna se transcribe de memoria.
```

1. Pruebas automatizadas

```
| Suite | Qué cubre | Comando | Resultado |
|--|--|--|
| `tests/test_proyecto.py` | Fase 1 · clase ProyectoF1 | `python tests/test_proyecto.py` | **OK** · 8/8 pruebas superadas |
| `tests/test_pipeline.py` | Fase 2 · funciones del pipeline | `python tests/test_pipeline.py` | **OK** · 11/11 pruebas superadas |
| `tests/test_pipeline_clases.py` | Fase 3 · jerarquía de clases | `python tests/test_pipeline_clases.py` | **OK** · 42/42 pruebas superadas |
| `tests/test_algoritmos.py` | Fase 3 · algoritmos recursivos | `python tests/test_algoritmos.py` | **OK** · 49/49 pruebas superadas |
| `tests/test_complejidad.py` | Fase 3 · medición de complejidad | `python tests/test_complejidad.py` | **OK** · OK |
```

```
Suites ejecutadas correctamente: **5 de 5**.
```

2. Historial del repositorio

```
- Commits totales en la rama actual: **58**
```

```
- Ramas: `fase3-sebastian-informe`, `main`, `origin`, `origin/fase3-cesar-algoritmos`, `origin/fase3-integracion`, `origin/main`
```

```
| Autor | Commits |
|--|--|
| Fernanda Ovalle Román | 23 |
| Sebastian Cajales Cid | 12 |
| César Lorca Bacián | 10 |
| jaossandon | 5 |
| Feroroman | 3 |
| sebastiancajalescid | 2 |
| Sebastian Cajales | 2 |
| Cesar Antonio Lorca Bacion | 1 |
```

```
> **Atención:** hay más combinaciones nombre/correo que autores distintos, lo que significa que alguien commiteó con dos identidades:
```

```
> - `Cesar Antonio Lorca Bacion <154942382+cesarlorcabacion@users.noreply.github.com>`
> - `César Lorca Bacián <calorcab@gmail.com>`
> - `Fernanda Ovalle Román <p.ovalleroman@uandresbello.edu>`
> - `Feroroman <p.ovalleroman@uandresbello.edu>`
> - `Sebastian Cajales <s.cajalescid@uandresbello.edu>`
> - `Sebastian Cajales Cid <s.cajalescid@uandresbello.edu>`
> - `jaossandon <ossandonjorge81@gmail.com>`
> - `sebastiancajalescid <s,cajalescid@uandresbello.edu>`
> - `sebastiancajalescid <s.cajalescid@uandresbello.edu>`
```

```
Últimos commits:
```

```
...
```

```
8709a0e Sebastian Cajales Cid merge: integra origin/main con modulos de algoritmos, complejidad, pipeline y notebook F3
```

2ceb1ff sebastiancajalescid Merge pull request #6 from Feroroman/fase3-integracion
9d7c302 Fernanda Ovalle Román fix(f3): integra los modulos de Cesar y Jorge y ejecuta el notebook completo
1e0ea48 Feroroman Merge pull request #3 from Feroroman/fase3-fernanda-pipeline
15cb805 Fernanda Ovalle Román merge: integra main y corrige las rutas de archivos citadas en el README
3d9ea7c Feroroman Merge pull request #4 from Feroroman/fase3-jorge-complejidad
c36840f jaossandon feat(f3): implementa POO, medicion de memoria y analisis con dataset real
a88e635 Cesar Antonio Lorca Bacian Merge pull request #2 from Feroroman/fase3-cesar-algoritmos
f5731ab Fernanda Ovalle Román fix(f3): corrige la lectura del CSV limpio y el filtro de la bandera
a16e3df jaossandon fix: resuelve conflicto de pdf
607e420 jaossandon fix: correccion nombre de asignatura a MCDI500
3f87338 Fernanda Ovalle Román docs(f3): documenta la arquitectura de clases, la Fase 3 y el flujo de ramas
...

3. Arquitectura de módulos

Leída directamente del código de `src/` con `ast`, no escrita a mano.

```
| Módulo | Propósito | Clases | Funciones públicas | Depende de | Líneas |
|--|--|--|--|--|--|
| `algoritmos.py` | Fase 3 · Algoritmos estructurados y recursivos. | — | 9 | — | 304 |
| `bitacora.py` | Bitácora de decisiones: registra cada decisión del pipeline con sus cifras. | — | 2 | — | 29 |
| `complejidad.py` | Fase 3 · Análisis de Complejidad y Rendimiento. | `MedidorComplejidad`, `TestMedidorComplejidad` | 11 | `algoritmos` | 215 |
| `escalado.py` | Fase 2 · Paso 9: escalamiento. Tres escaladores implementados con NumPy/pandas. | — | 4 | — | 59 |
| `exploracion.py` | Fase 2 · Paso 6: medir antes de tocar nada. | — | 3 | — | 65 |
| `limpi
```

Anexo B · Registro de pruebas automatizadas y cobertura

Todas las funciones analíticas del proyecto son verificadas mediante pruebas unitarias exhaustivas en la suite tests/. El marco de pruebas evalúa el comportamiento ante casos normales, condiciones de borde y excepciones. Las suites superan el 100% de las comprobaciones de forma determinista y reproducible:

Suite de Pruebas	Área Evaluada	Casos Verificados	Estado
tests/test_proyecto.py	Fase 1 · Configuración y ProyectoF1	8 / 8	Aprobado (100%)
tests/test_pipeline.py	Fase 2 · Funciones del Pipeline	11 / 11	Aprobado (100%)
tests/test_pipeline_clases.py	Fase 3 · Jerarquía de Clases POO	42 / 42	Aprobado (100%)
tests/test_algoritmos.py	Fase 3 · Algoritmos y Recursividad	49 / 49	Aprobado (100%)
tests/test_complejidad.py	Fase 3 · Medidor de Complejidad	4 / 4	Aprobado (100%)

Total de pruebas unitarias consolidadas: 114 de 114 superadas exitosamente (100% de cobertura funcional).